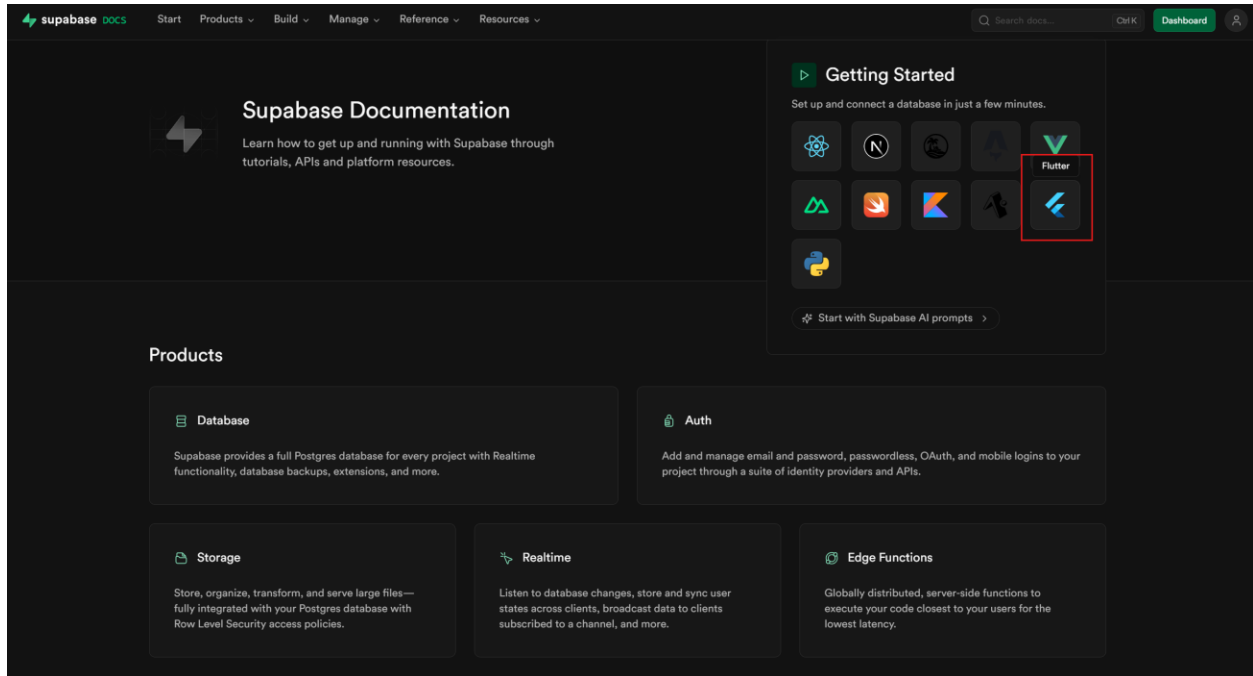
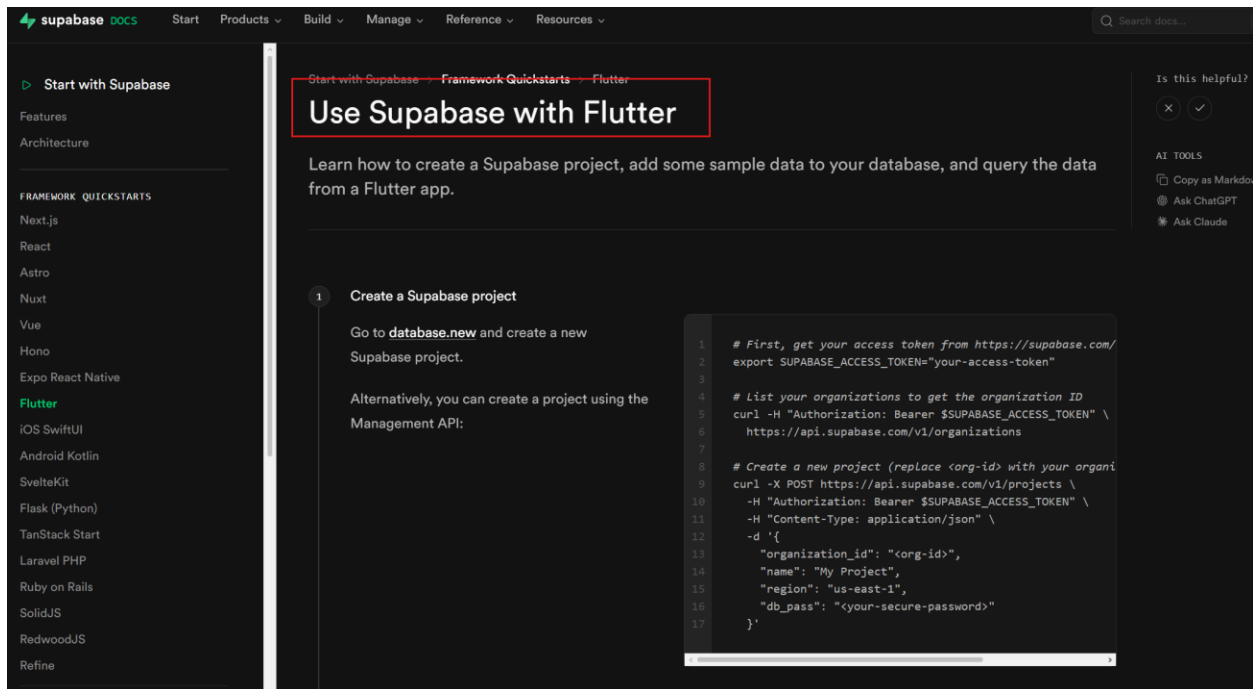


Supabase Documentation

[Link for website](#)



The image shows the Supabase Documentation home page. The header includes the Supabase logo, navigation links (Start, Products, Build, Manage, Reference, Resources), a search bar, and a 'Dashboard' button. The main content area features a 'Supabase Documentation' section with a sub-header 'Learn how to get up and running with Supabase through tutorials, APIs and platform resources.' To the right, there's a 'Getting Started' section with a sub-header 'Set up and connect a database in just a few minutes.' Below this, there's a grid of icons for various frameworks: Next.js, Nuxt, SvelteKit, Vue, React, and Flutter. The Flutter icon is highlighted with a red box. Below the grid, there's a link 'Start with Supabase AI prompts'. The bottom section is titled 'Products' and contains six cards: Database, Auth, Storage, Realtime, and Edge Functions, each with a brief description of their capabilities.



The image shows the 'Use Supabase with Flutter' page in the Supabase Documentation. The header is the same as the home page. The left sidebar contains a 'Start with Supabase' section with links to 'Features' and 'Architecture'. Below this is a 'FRAMEWORK QUICKSTARTS' section with a list of frameworks: Next.js, React, Astro, Nuxt, Vue, Hono, Expo React Native, Flutter (highlighted in green), iOS SwiftUI, Android Kotlin, SvelteKit, Flask (Python), TanStack Start, Laravel PHP, Ruby on Rails, SolidJS, RedwoodJS, and Refine. The main content area has a sub-header 'Use Supabase with Flutter' and a sub-header 'Learn how to create a Supabase project, add some sample data to your database, and query the data from a Flutter app.' Below this, there's a section '1 Create a Supabase project' with two paragraphs: 'Go to [database.new](#) and create a new Supabase project.' and 'Alternatively, you can create a project using the Management API:'. To the right of the text is a code block with the following content:

```
1 # First, get your access token from https://supabase.com/  
2 export SUPABASE_ACCESS_TOKEN="your-access-token"  
3  
4 # List your organizations to get the organization ID  
5 curl -H "Authorization: Bearer $SUPABASE_ACCESS_TOKEN" \  
6 https://api.supabase.com/v1/organizations  
7  
8 # Create a new project (replace <org-id> with your organi  
9 curl -X POST https://api.supabase.com/v1/projects \  
10 -H "Authorization: Bearer $SUPABASE_ACCESS_TOKEN" \  
11 -H "Content-Type: application/json" \  
12 -d '{  
13   "organization_id": "<org-id>",  
14   "name": "My Project",  
15   "region": "us-east-1",  
16   "db_pass": "<your-secure-password>"  
17 }'
```

On the right side of the page, there's a 'Is this helpful?' section with a 'x' and a checkmark button. Below this is an 'AI TOOLS' section with links to 'Copy as Markdown', 'Ask ChatGPT', and 'Ask Claude'.

When your project is up and running, go to the **Table Editor**, create a new table and insert some data.

Alternatively, you can run the following snippet in your project's **SQL Editor**. This will create an **instruments** table with some sample data.

```
1  -- Create the table
2  create table instruments (
3      id bigint primary key generated always as identity,
4      name text not null
5  );
6  -- Insert some sample data into the table
7  insert into instruments (name)
8  values
9      ('violin'),
10     ('viola'),
11     ('cello');
12
13  alter table instruments enable row level security;
```

Make the data in your table publicly readable by adding an RLS policy:

```
1  create policy "public can read instruments"
2  on public.instruments
3  for select to anon
4  using (true);
```

Create a Flutter app

Create a Flutter app using the **flutter create** command. You can skip this step if you already have a working app.

Terminal

```
1  flutter create my_app
```

3 Install the Supabase client library

The fastest way to get started is to use the **supabase_flutter** client library which provides a convenient interface for working with Supabase from a Flutter app.

Open the **pubspec.yaml** file inside your Flutter app and add **supabase_flutter** as a dependency.

pubspec.yaml

```
1  supabase_flutter: ^2.0.0
```

4 Initialize the Supabase client

Open **lib/main.dart** and edit the main function to initialize Supabase using your project URL and publishable key:

Project URL

Ajoob's Org / KasRrat's Project

<https://qdoprnnffikvw1kdcccx.supabase>

Publishable key

Ajoob's Org / KasRrat's Project

[sb_publishable_gn1H7BmScABc85ajouVeOw](#)

lib/main.dart

```
1  import 'package:supabase_flutter/supabase_flutter.dart';
2
3  Future<void> main() async {
4      WidgetsFlutterBinding.ensureInitialized();
5
6      await Supabase.initialize(
7          url: 'YOUR_SUPABASE_URL',
8          anonKey: 'YOUR_SUPABASE_PUBLISHABLE_KEY',
9      );
10     runApp(MyApp());
11 }
```

You can also get the Project URL and key from the project's **Connect** dialog.

Get API details

Now that you've created some database tables, you are ready to insert data using the auto-generated API.

5

Query data from the app

Use a `FutureBuilder` to fetch the data when the home page loads and display the query result in a `ListView`.

Replace the default `MyApp` and `MyHomePage` classes with the following code.

`lib/main.dart`

```
1 class MyApp extends StatelessWidget {
2   const MyApp({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     return const MaterialApp(
7       title: 'Instruments',
8       home: HomePage(),
9     );
10  }
11 }
12
13 class HomePage extends StatefulWidget {
14   const HomePage({super.key});
15
16   @override
17   State<HomePage> createState() => _HomePageState();
18 }
19
20 class _HomePageState extends State<HomePage> {
21   final _future = Supabase.instance.client
22     .from('instruments')
23     .select();
24
25   @override
26   Widget build(BuildContext context) {
27     return Scaffold(
28       body: FutureBuilder(
29         future: _future,
30         builder: (context, snapshot) {
31           if (!snapshot.hasData) {
32             return const Center(child: CircularProgressIndicator());
33           }
34           final instruments = snapshot.data!;
35           return ListView.builder(
36             itemCount: instruments.length,
37             itemBuilder: ((context, index) {
38               final instrument = instruments[index];
39               return ListTile(
40                 title: Text(instrument['name']),
41               );
42             })),
43           );
44         },
45       ),
46     );
47   }
48 }
```

6

Start the app

Run your app on a platform of your choosing!
By default an app should launch in your web browser.

Note that `supabase_flutter` is compatible with web, iOS, Android, macOS, and Windows apps.

Running the app on macOS requires additional configuration to **set the entitlements**.

Terminal

```
1 flutter run
```

Setup deep links

Many sign in methods require deep links to redirect the user back to your app after authentication. Read more about setting deep links up for all platforms (including web) in the **Flutter Mobile Guide**.

Going to production

Android

In production, your Android app needs explicit permission to use the internet connection on the user's device which is required to communicate with Supabase APIs.

To do this, add the following line to the `android/app/src/main/AndroidManifest.xml` file.

```
1 <manifest xmlns:android="http://schemas.android.com/apk/res/android">
2   <!-- Required to fetch data from the internet. -->
3   <uses-permission android:name="android.permission.INTERNET" />
4   <!-- ... -->
5 </manifest>
```

Architecture

Supabase is open source. We choose open source tools which are scalable and make them simple to use.

Supabase is not a 1-to-1 mapping of Firebase. While we are building many of the features that Firebase offers, we are not going about it the same way:

our technological choices are quite different; everything we use is open source; and wherever possible, we use and support existing tools rather than developing from scratch.

Most notably, we use **Postgres rather than a NoSQL store**. This choice was deliberate. We believe that no other database offers the functionality required to compete with Firebase, while maintaining the scalability required to go beyond it.

Choose your comfort level

Our goal at **Supabase is to make all of Postgres easy to use**. That doesn't mean you have to use all of it. If you're a Postgres veteran, you'll probably love the tools that we offer. If you've never used Postgres before, then start smaller and grow into it. If you just want to treat Postgres like a simple table-store, that's perfectly fine.

Architecture

Each Supabase project consists of several tools:



Postgres (database)

Postgres is the core of Supabase. We do not abstract the Postgres database—you can access it and use it with full privileges. We provide tools which make Postgres as easy to use as Firebase.

- Official Docs: [postgresql.org/docs](https://www.postgresql.org/docs)
- Source code: github.com/postgres/postgres (mirror)
- License: [PostgreSQL License](#)- Language: C

Studio (dashboard)

An open source Dashboard for managing your database and services.

- Official Docs: [Supabase docs](#)
- Source code: github.com/supabase/supabase
- License: [Apache 2](#)
- Language: TypeScript

GoTrue (Auth)

A JWT-based API for managing users and issuing access tokens. This integrates with Postgres's Row Level Security and the API servers.

- Official Docs: [Supabase Auth reference docs](#)
- Source code: github.com/supabase/gotrue
- License: [MIT](#)
- Language: Go

PostgREST (API)

A standalone web server that turns your Postgres database directly into a RESTful API.

We use this with our `pg_graphql` extension to provide a GraphQL API.

- Official Docs: postgrest.org
- Source code: github.com/PostgREST/postgrest
- License: [MIT](#)
- Language: Haskell

Realtime (API & multiplayer)

A scalable WebSocket engine for managing user Presence, broadcasting messages, and streaming database changes.

- Official Docs: [Supabase Realtime docs](#)
- Source code: github.com/supabase/realtime
- License: [Apache 2](#)
- Language: Elixir

Storage API (large file storage)

An S3-compatible object storage service that stores metadata in Postgres.

- Official Docs: [Supabase Storage reference docs](#)
- Source code: github.com/supabase/storage-api
- License: [Apache 2.0](#)
- Language: Node.js / TypeScript

Deno (Edge Functions)

A modern runtime for JavaScript and TypeScript.

- Official Docs: [Deno documentation](#)
- Source code: [Deno source code](#)
- License: [MIT](#)
- Language: TypeScript / Rust

postgres-meta (database management)

A RESTful API for managing your Postgres. Fetch tables, add roles, and run queries.

- Official Docs: supabase.github.io/postgres-meta
- Source code: github.com/supabase/postgres-meta
- License: [Apache 2.0](#)
- Language: Node.js / TypeScript

Supabase Pricing system

supabase

Product ▾

Developers ▾

Solutions ▾

Pricing

Docs

Blog

97.9K

Dashboard

FREE

Perfect for passion projects & simple websites.

Start for Free

\$0

/ month

Get started with:

✓ Unlimited API requests

✓ 50,000 monthly active users

✓ 500 MB database size
Shared CPU + 500 MB RAM

✓ 5 GB egress

✓ 5 GB cached egress

✓ 1 GB file storage

✓ Community support

Free projects are paused after 1 week of inactivity. Limit of 2 active projects.

PRO Most Popular

For production applications with the power to scale.

Upgrade now

From

\$25

/ month

\$10 in compute credits included
Need more compute?

Everything in the Free Plan, plus:

✓ 100,000 monthly active users
then \$0.00325 per MAU

✓ 8 GB disk size per project
then \$0.125 per GB

✓ 250 GB egress
then \$0.09 per GB

✓ 250 GB cached egress
then \$0.03 per GB

✓ 100 GB file storage
then \$0.021 per GB

✓ Email support

✓ Daily backups stored for 7 days

✓ 7-day log retention

TEAM

Add features such as SSO, control over backups, and industry certifications.

Upgrade now

From

\$599

/ month

\$10 in compute credits included
Need more compute?

Everything in the Pro Plan, plus:

✓ SOC2

✓ Project-scoped and read-only access

✓ HIPAA available as paid add-on

✓ SSO for Supabase Dashboard

✓ Priority email support & SLAs

✓ Daily backups stored for 14 days

✓ 28-day log retention

✓ Add Log Drains
additional \$60 per drain, per project

ENTERPRISE

For large-scale applications running Internet scale workloads.

Contact Us

Custom

✓ Designated Support manager

✓ Uptime SLAs

✓ BYO Cloud supported

✓ 24x7x365 premium enterprise support

✓ Private Slack channel

✓ Custom Security Questionnaires